

AFRILANG Stdlib

std/cache

Français. Bibliothèque AFRILANG 'std/cache' (niveau simple, catégorie system). Elle expose 2 fonction(s) : ttlExpired, freshness.

'import "std/cache"' · 'use cache'

- 'ttlExpired(ageMs number, ttlMs number) → bool' - 'freshness(ageMs number, ttlMs number) → number'

English. AFRILANG library 'std/cache' (tier simple, category system). Exports 2 function(s): ttlExpired, freshness.

'import "std/cache"' · 'use cache'

- 'ttlExpired(ageMs number, ttlMs number) → bool' - 'freshness(ageMs number, ttlMs number) → number'

Catégorie / Category	Système	
Niveau / Tier	simple	June 16, 2026
Fonctions / Functions	2	

Contents

Français

1. Vue d'ensemble

Bibliothèque AFRILANG 'std/cache' (niveau simple, catégorie system). Elle expose 2 fonction(s) : ttlExpired, freshness.

'import "std/cache"' · 'use cache'

```
- `ttlExpired(ageMs number, ttlMs number) → bool`
- `freshness(ageMs number, ttlMs number) → number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/cache"
use cache
```

Niveau : simple · Catégorie : Système
CPU, disque, processus, environnement.

3. Référence API

- ttlExpired(ageMs number, ttlMs number) → bool
freshness(ageMs number, ttlMs number) → number

4. Exemples d'utilisation

```
import "std/cache"
use cache
```

```
create result = ttlExpired(/* args */)
say result
```

```
create result = freshness(/* args */)
say result
```

5. Bonnes pratiques

- Préférez `import "std/cache"` en tête de fichier.
Lancez `afrilang check` avant de livrer.
Combinez avec les modules stdlib de la même catégorie.
Voir /stdlib/ pour le source live et les démos playground.
Signalez les problèmes sur GitHub si une export manque.

6. Annexe — source

module cache

```
function ttlExpired(ageMs number, ttlMs number) returns bool
end
```

```
function freshness(ageMs number, ttlMs number) returns number
end
end
```

English

1. Overview

AFRILANG library 'std/cache' (tier simple, category system). Exports 2 function(s): ttlExpired, freshness.

'import "std/cache"' · 'use cache'

```
- `ttlExpired(ageMs number, ttlMs number) → bool`
- `freshness(ageMs number, ttlMs number) → number`
```

2. Installation & import

Add the module to your program:

```
import "std/cache"
use cache
```

Tier: simple · Category: System
CPU, disk, processes, environment.

3. API reference

- ttlExpired(ageMs number, ttlMs number) → bool
- freshness(ageMs number, ttlMs number) → number

4. Usage examples

```
import "std/cache"
use cache
```

```
create result = ttlExpired(/* args */)
say result
```

```
create result = freshness(/* args */)
say result
```

5. Best practices

- Prefer `import "std/cache"` at the top of your file.
- Run `afri-lang check` before shipping.
- Combine with related stdlib modules from the same category.
- See /stdlib/ for the live source and playground demos.
- Report issues on GitHub if an export is missing or undocumented.

6. Appendix — source

module cache

```
function ttlExpired(ageMs number, ttlMs number) returns bool
end
```

```
function freshness(ageMs number, ttlMs number) returns number
end
end
```

Référence rapide / Quick reference

- Import : `import "std/cache"`
- Vérification : `afrilang check`
- Documentation : <https://afrilang.dev/stdlib/std/cache/>

Source (extrait)

```
module cache

function ttlExpired(ageMs number, ttlMs number) returns bool
end

function freshness(ageMs number, ttlMs number) returns number
end
end
```